

Flask

What is Flask?

Flask is a lightweight and flexible web framework for building web applications in Python

Key Features of Flask

Lightweight and Modular: Flask is designed to be minimal. It doesn't require a lot of boilerplate code to get started, and you can add extensions as needed.

Built-in Development Server: Flask comes with a built-in server for testing your application during development.

Jinja2 Templating: Flask uses Jinja2 as its template engine, which allows you to separate your logic from your HTML pages.

Request Handling: Flask provides easy-to-use functions to handle HTTP requests and routing.

Extensibility: There are many Flask extensions that allow you to add database integration, form handling, authentication, and more.

Library Management System Using Flask

- Create a new folder in C Drive “Library Management System
- Open Anaconda Command Prompt

Create Virtual Environment

```
(base) C:\Users\Datamites>cd..  
(base) C:\Users>cd..  
(base) C:\>cd Library Management System  
(base) C:\Library Management System>conda create --name myflask python==3.9
```

Activate Virtual Environment

```
(base) C:\Library Management System>conda activate myflask
```

Install flask

```
(myflask) C:\Library Management System>pip install flask
```

Visual Studio Code Installation

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ **Windows**

Windows 10, 11

User Installer	x64	Arm64
System Installer	x64	Arm64
.zip	x64	Arm64
CLI	x64	Arm64



↓ **.deb**

Debian, Ubuntu

↓ **.rpm**

Red Hat, Fedora, SUSE

.deb	x64	Arm32	Arm64
.rpm	x64	Arm32	Arm64
.tar.gz	x64	Arm32	Arm64
Snap	Snap Store		
CLI	x64	Arm32	Arm64



↓ **Mac**

macOS 10.15+

.zip	Intel chip	Apple silicon	Universal
CLI	Intel chip	Apple silicon	

- Open Library Management System folder in VS code
- Create a new file **app.py** in this folder

An **app.py** file in Flask is the main entry point of a Flask web application, which initializes the app, sets up routes, and handles incoming requests.

In app.py, Initialize SQL database

```
from flask import Flask, render_template, request, redirect, url_for, flash
import sqlite3

app = Flask(__name__)
app.secret_key = "secretkey"

# Initialize the SQLite database
def init_sqlite_db():
    conn = sqlite3.connect('library.db')
    conn.execute('CREATE TABLE IF NOT EXISTS books (id INTEGER PRIMARY KEY, title TEXT, author TEXT, year INTEGER)')
    conn.close()

init_sqlite_db()
```

Display All Books

```
# Home Page - Display All Books
@app.route('/')
def index():
    conn = sqlite3.connect('library.db')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM books")
    books = cursor.fetchall()
    conn.close()
    return render_template('index.html', books=books)
```

Add New Books

```
# Add a New Book
@app.route('/add_book', methods=['GET', 'POST'])
def add_book():
    if request.method == 'POST':
        title = request.form['title']
        author = request.form['author']
        year = request.form['year']

        if title and author and year:
            conn = sqlite3.connect('library.db')
            cursor = conn.cursor()
            cursor.execute("INSERT INTO books (title, author, year) VALUES (?, ?, ?)", (title, author, year))
            conn.commit()
            conn.close()
            flash('Book added successfully!')
            return redirect(url_for('index'))
        else:
            flash('All fields are required!')
    return render_template('add_book.html')
```

Update Book Details

```
# Update Book Details
@app.route('/update_book/<int:id>', methods=['GET', 'POST'])
def update_book(id):
    conn = sqlite3.connect('library.db')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    book = cursor.fetchone()
    conn.close()

    if request.method == 'POST':
        title = request.form['title']
        author = request.form['author']
        year = request.form['year']

        if title and author and year:
            conn = sqlite3.connect('library.db')
            cursor = conn.cursor()
            cursor.execute("UPDATE books SET title = ?, author = ?, year = ? WHERE id = ?", (title, author, year, id))
            conn.commit()
            conn.close()
            flash('Book updated successfully!')
            return redirect(url_for('index'))
        else:
            flash('All fields are required!')

    return render_template('update_book.html', book=book)
```

Delete a book

```
# Delete a Book
@app.route('/delete_book/<int:id>', methods=['GET'])
def delete_book(id):
    conn = sqlite3.connect('library.db')
    cursor = conn.cursor()
    cursor.execute("DELETE FROM books WHERE id = ?", (id,))
    conn.commit()
    conn.close()
    flash('Book deleted successfully!')
    return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(debug=True)
```

Create base.html in templates folder

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Library Management System</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <header>
    <h1>Library Management System</h1>
    <nav>
      <a href="{{ url_for('index') }}">Home</a> |
      <a href="{{ url_for('add_book') }}">Add Book</a>
    </nav>
  </header>
  <main>
    {% with messages = get_flashed_messages() %}
    {% if messages %}
      <ul>
        {% for message in messages %}
          <li>{{ message }}</li>
        {% endfor %}
      </ul>
    {% endif %}
    {% endif %}
    {% block content %}
  </main>
</body>
</html>
```

Create index.html file in templates folder

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
<h2>Book List</h2>
```

```
<table>
```

```
<tr>
```

```
<th>ID</th>
```

```
<th>Title</th>
```

```
<th>Author</th>
```

```
<th>Year</th>
```

```
<th>Actions</th>
```

```
</tr>
```

```
{% for book in books %}
```

```
<tr>
```

```
<td>{{ book[0] }}</td>
```

```
<td>{{ book[1] }}</td>
```

```
<td>{{ book[2] }}</td>
```

```
<td>{{ book[3] }}</td>
```

```
<td>
```

```
<a href="{{ url_for('update_book', id=book[0]) }}">Edit</a> |
```

```
<a href="{{ url_for('delete_book', id=book[0]) }}">Delete</a>
```

```
</td>
```

```
</tr>
```

```
{% endfor %}
```

```
</table>
```

```
{% endblock %}
```

Create add_book.html in templates folder

```
{% extends 'base.html' %}

{% block content %}
<h2>Add a New Book</h2>
<form method="POST">
    <label for="title">Title:</label>
    <input type="text" id="title" name="title" required>

    <label for="author">Author:</label>
    <input type="text" id="author" name="author" required>

    <label for="year">Year:</label>
    <input type="number" id="year" name="year" required>

    <button type="submit">Add Book</button>
</form>
{% endblock %}
```

Create update_book. html in templates folder

```
{% extends 'base.html' %}

{% block content %}
<h2>Update Book</h2>
<form method="POST">
    <label for="title">Title:</label>
    <input type="text" id="title" name="title" value="{{ book[1] }}" required>

    <label for="author">Author:</label>
    <input type="text" id="author" name="author" value="{{ book[2] }}" required>

    <label for="year">Year:</label>
    <input type="number" id="year" name="year" value="{{ book[3] }}" required>

    <button type="submit">Update Book</button>
</form>
{% endblock %}
```

Create a folder static & create style.css in it to add style to website

```
body {  
  font-family: Arial, sans-serif;  
  margin: 0;  
  padding: 0;  
  background-color: #f4f4f4;  
}
```

```
header {  
  background-color: #4CAF50;  
  color: white;  
  padding: 1rem;  
  text-align: center;  
}
```

```
nav a {  
  color: white;  
  margin: 0 15px;  
  text-decoration: none;  
}
```

```
table {  
  width: 100%;  
  border-collapse: collapse;  
  margin-top: 20px;  
}
```

```
table, th, td {  
  border: 1px solid #ddd;  
  padding: 8px;  
}
```

```
table th {  
  background-color: #4CAF50;  
  color: white;  
}
```

Install **SQLite Viewer** extension to view SQL Database

- Next Open command prompt (*same one where you have activated virtual environment*)
- Run the Flask app by executing:

```
(myflask) C:\Library Management System>python app.py
```

- Open the browser and go to <http://127.0.0.1:5000> to view the Library Management System

Library Management System

[Home](#) | [Add Book](#)

Book List

ID	Title	Author	Year	Actions



127.0.0.1:5000/add_book

Library Management System

[Home](#) | [Add Book](#)

Add a New Book

Title: Author: Year:

Library Management System

[Home](#) | [Add Book](#)

Add a New Book

Title: Author: Year:

Library Management System

[Home](#) | [Add Book](#)

- Book added successfully!

Book List

ID	Title	Author	Year	Actions
1	Harry Potter	J. K. Rowling	2000	Edit Delete

Library Management System

[Home](#) | [Add Book](#)

Update Book

Title: Author: Year:

Library Management System

[Home](#) | [Add Book](#)

- Book updated successfully!

Book List

ID	Title	Author	Year	Actions
1	Harry Potter	J. K. Rowling	1998	Edit Delete

Library Management System

[Home](#) | [Add Book](#)

- Book deleted successfully!

Book List

ID	Title	Author	Year	Actions

*Thank
you*